Le modèle Dreamer v3

Fiche de révision pour la soutenance
Projet *Order Book Engine* – ENSEIRB-MATMECA

Yani Haouchine

7 mai 2026

Résumé

Ce document récapitule *tout ce qu'il faut savoir* sur l'agent Dreamer v3 implémenté dans le projet pour le *trading algorithmique* : principes de l'apprentissage par renforcement basé sur un modèle du monde, architecture neuronale (Encoder, RSSM, Decoder, Reward/Continue, Actor, Critic), procédure d'entraînement en trois phases, astuces numériques propres à Dreamer v3 (symlog, free bits, KL balancing, percentile normalization, EMA target critic, unimix exploration), résultats obtenus sur les données `SOLUSDT` de Binance, et liste des questions susceptibles d'être posées en soutenance avec leurs réponses.

Table des matières

1 Pourquoi Dreamer v3 ? 1
1.1 Le problème : RL classique sur des séries financières . . . 1
1.2 L'idée de Dreamer v3 : apprendre à *rêver* . . . 1
1.3 Place de Dreamer v3 dans l'état de l'art . . . 2

2 Vue d'ensemble de l'architecture 2
2.1 Composants . . . 2
2.2 Schéma d'ensemble . . . 2
2.3 État latent : $s_t = (h_t, z_t)$ . . . 2

3 Le RSSM en détail 2
3.1 Mode *posterior* (avec observation) . . . 3
3.2 Mode *imagine* (sans observation) . . . 3
3.3 Latents catégoriels et *straight-through* . . . 3

4 Procédure d'entraînement 3
4.1 Phase 1 – Collecte d'un épisode réel . . . 3
4.2 Phase 2a – Entraînement du modèle du monde . . . 3
4.3 Phase 2b – Entraînement acteur-critique *en imagination* . . . 4
4.4 Hyperparamètres utilisés . . . 4

5 Les six *tricks* de Dreamer v3 v3 4
5.1 1. Symlog / symexp . . . 4
5.2 2. Free bits sur le KL . . . 4
5.3 3. KL balancing . . . 5
5.4 4. Percentile normalization des returns . . . 5
5.5 5. EMA target critic . . . 5
5.6 6. Unimix exploration . . . 5

6 Application au *trading* crypto **5**
6.1 Environnement . . . 5
6.2 Actions discrètes . . . 5
6.3 Récompense . . . 5
6.4 Inférence en production . . . 6
6.5 Résultats . . . 6

7 Forces et limites **6**
7.1 Forces . . . 6
7.2 Limites . . . 7

8 Comparaison avec les autres agents du projet **7**

9 Code clé : extraits commentés **7**
9.1 Boucle d'entraînement (cœur) . . . 7
9.2 Calcul des λ-returns (rétropropagation arrière) . . . 7

10 Questions probables en soutenance (& réponses) **8**

11 Pour aller plus loin **8**

1 Pourquoi Dreamer v3 ?

1.1 Le problème : RL classique sur des séries financières

Dans le projet, plusieurs agents *model-free* sont déjà implémentés (**DQN**, **A2C**, **PPO**, **SAC**). Ils partagent un même défaut : ils apprennent uniquement par essais / erreurs sur l'environnement réel. Pour le trading, cela pose deux problèmes :

— **Inefficacité en données.** Une époque = un parcours complet du fichier CSV. Les transitions sont consommées une seule fois si le replay buffer est petit.

— **Exploration coûteuse.** Toute mauvaise action exécutée se paie en pertes simulées. Sur des marchés volatils, le signal/bruit est très faible.

1.2 L'idée de Dreamer v3 : apprendre à *rêver*

Dreamer v3 appartient à la famille des **model-based RL**. Il apprend deux choses simultanément :

1. Un **modèle du monde** (world model) qui prédit la dynamique de l'environnement : à partir d'un état latent et d'une action, il prédit l'observation suivante, la récompense et un drapeau de fin d'épisode.
2. Une politique acteur-critique qui s'entraîne **exclusivement dans l'imagination du modèle**, sans interagir avec l'environnement réel pendant cette phase.

L'avantage est double : le modèle peut être entraîné sur des séquences rejouées indéfiniment, et l'acteur génère des trajectoires fictives très peu coûteuses (un simple rollout neuronal).

1.3 Place de Dreamer v3 dans l'état de l'art

— **DreamerV1** (Hafner et al., 2020) – POMDP visuels.

— **DreamerV2** (2021) – premier modèle à battre Rainbow sur Atari avec un seul GPU.

— **DreamerV3** (2023) – la version implémentée ici. Mêmes hyperparamètres pour 150+ environnements (Atari, DMLab, Crafter, Minecraft…), grâce aux astuces numériques que nous décrivons plus bas.

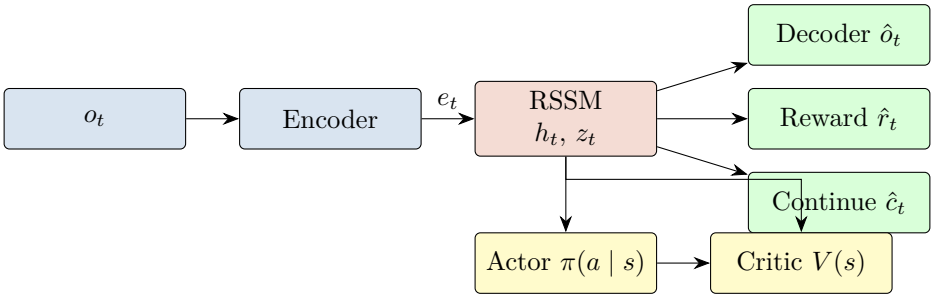
2 Vue d'ensemble de l'architecture

2.1 Composants

L'agent est composé de **sept réseaux de neurones** regroupés en deux blocs.

Bloc	Réseau (rôle)
Modèle du monde	**Encoder** – compresse l'observation en un embedding **RSSM** – modèle récurrent à variables latentes catégorielles **Decoder** – reconstruit l'observation depuis l'état latent **Reward predictor** – prédit la récompense **Continue predictor** – prédit le drapeau de fin (continue/done)
Acteur-Critique	**Actor** – politique $\pi(a \mid s)$, softmax sur 3 actions **Critic** – fonction valeur $V(s)$, entraînée par EMA target

2.2 Schéma d'ensemble



2.3 État latent : $s_t = (h_t, z_t)$

À chaque pas de temps, l'état est la *concaténation* de :

— h_t (**déterministe**, dim 512) – mémoire récurrente, sortie de la GRU.

— z_t (**stochastique**, dim $1024 = 32 \times 32$) – variables latentes *catégorielles*, échantillonnées via straight-through estimator.

Donc $\dim(s_t) = 1536$. Cet état alimente *tous* les prédicteurs ainsi que l'acteur et le critique.

3 Le RSSM en détail

Le *Recurrent State-Space Model* est le cœur de Dreamer v3. C'est lui qui apprend à *simuler le marché*.

3.1 Mode *posterior* (avec observation)

À l'entraînement du modèle du monde, on observe o_t. On calcule :

$$e_t = \text{Encoder}(o_t) \tag{1}$$

$$h_t = \text{GRU}(\text{Lin}([z_{t-1}, a_{t-1}]),\ h_{t-1}) \tag{2}$$

$$p(z_t \mid h_t) = \text{Cat}(\text{prior_net}(h_t)) \quad \text{[prior]} \tag{3}$$

$$q(z_t \mid h_t, e_t) = \text{Cat}(\text{post_net}([h_t, e_t])) \quad \text{[posterior]} \tag{4}$$

On échantillonne $z_t \sim q(\cdot)$ et on conserve les deux distributions pour la perte KL.

3.2 Mode *imagine* (sans observation)

Pendant l'entraînement de l'acteur (phase 2b), on ne dispose plus de o_t. On utilise donc le *prior* seul :

$$h_t = \mathrm{GRU}(\mathrm{Lin}([z_{t-1}, a_{t-1}]),\ h_{t-1}) \tag{5}$$

$$z_t \sim p(z_t \mid h_t) \tag{6}$$

C'est la *simulation pure* : tout ce qui sort vient du modèle.

3.3 Latents catégoriels et *straight-through*

Les z_t sont 32 catégorielles à 32 classes (au lieu d'une gaussienne diagonale comme dans DreamerV1). Avantages :

— Représentation discrète ⇒ plus expressive et plus stable que des gaussiennes.

— Le KL entre catégorielles est calculé exactement.

Pour rétropropager le gradient à travers l'échantillon, on utilise le **straight-through estimator** :

$$z = \underbrace{\mathrm{onehot}(\arg\max\ p)}_{\text{forward}} + \underbrace{p - \mathrm{stop_grad}(p)}_{\text{trick gradient}} \tag{7}$$

Le forward est exactement un one-hot, mais le backward voit la dérivée des probabilités softmax.

4 Procédure d'entraînement

À chaque **époque**, on déroule une boucle en trois temps :

4.1 Phase 1 – Collecte d'un épisode réel

1. `env.reset()` $\rightarrow o_0$.
2. Tant que non terminé :
   — $e_t = \mathrm{Encoder}(o_t)$,
   — $h_t, z_t = RSSM.\mathrm{observe_step}(h_{t-1}, z_{t-1}, a_{t-1}, e_t)$,
   — $a_t \sim \pi(\cdot \mid s_t)$ (catégorielle),
   — $o_{t+1}, r_t, \mathrm{done} \leftarrow \mathrm{env.step}(a_t)$.
3. Stocker l'épisode complet dans le *Sequential Replay Buffer*.

4.2 Phase 2a – Entraînement du modèle du monde

On échantillonne $B = 16$ sous-séquences de $T = 64$ pas. On déroule le RSSM en mode *posterior*, puis on minimise :

$$\boxed{\mathcal{L}_{\mathrm{WM}} = \underbrace{\mathcal{L}_{\mathrm{recon}}}_{\text{MSE sur } \hat{o}_t} + \underbrace{\mathcal{L}_{\mathrm{rew}}}_{\text{MSE sur symlog}(r_t)} + \underbrace{\mathcal{L}_{\mathrm{cont}}}_{\text{BCE sur } c_t} + \underbrace{\mathcal{L}_{\mathrm{KL}}}_{\text{cf. § KL balancing}}} \tag{8}$$

4.3 Phase 2b – Entraînement acteur-critique *en imagination*

À partir de chaque état s_t du batch (donc $N = B \cdot T$ trajectoires en parallèle !), on déroule le RSSM en mode *imagine* sur $H = 15$ **pas** :

— Échantillonner $a_h \sim \pi(\cdot \mid s_h)$.

— Prédire $\hat{r}_h$, $\hat{c}_h$, passer en s_{h+1}.

On obtient une trajectoire purement simulée. On calcule alors les **λ-returns** :

$$G_h^\lambda = \hat{r}_h + \gamma\, \hat{c}_h \left[(1-\lambda)\, V_\theta(s_{h+1}) + \lambda\, G_{h+1}^\lambda\right], \qquad G_H^\lambda = \hat{r}_H + \gamma \hat{c}_H V_\theta(s_H). \tag{9}$$

Critic. Régression sur symlog(G^λ) : $\mathcal{L}_{\text{critic}} = \text{MSE}(V_\theta(s_h),\ \text{symlog}(G_h^\lambda))$.

Actor. REINFORCE avec avantage normalisé et bonus d'entropie :

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}\big[A_h^{\text{norm}} \cdot \log \pi(a_h \mid s_h) + \beta\, \mathcal{H}[\pi(\cdot \mid s_h)]\big], \qquad \beta = 3 \cdot 10^{-4}. \tag{10}$$

4.4 Hyperparamètres utilisés

Paramètre	Symbole	Valeur
Époques d'entraînement	—	2000
Taille de batch	B	16
Longueur de séquence	T	64
Horizon d'imagination	H	15
Pas de gradient/époque	—	4
Facteur de discount	γ	0.997
Coefficient λ-return	λ	0.95
Free bits (KL)	—	1.0 nat
KL balancing	α	0.8
Unimix exploration	—	0.01
EMA target critic	—	0.98
Lr modèle du monde	—	10^{-4}
Lr acteur / critique	—	$3 \cdot 10^{-5}$
Coef. entropie	β	$3 \cdot 10^{-4}$
Clip de gradient	—	100.0

5 Les six *tricks* de Dreamer v3 v3

Ces astuces sont *ce qui rend Dreamer v3 v3 robuste* sans avoir à ajuster les hyperparamètres pour chaque domaine. Toutes sont dans le code du projet.

5.1 1. Symlog / symexp

Pour les récompenses et les valeurs, on apprend dans un espace compressé symétrique :

$$\text{symlog}(x) = \text{sign}(x)\ \log(1 + |x|), \qquad \text{symexp}(x) = \text{sign}(x)(\exp(|x|) - 1). \tag{11}$$

Cela permet d'avoir des gradients stables même quand les récompenses varient sur plusieurs ordres de grandeur (typique en finance !).

5.2 2. Free bits sur le KL

Le terme KL est borné par une valeur plancher (1 nat) avant moyennage : max(KL, 1.0). Empêche le posterior de coller artificiellement au prior et garantit toujours un peu d'information dans z_t.

5.3 3. KL balancing

Au lieu d'une seule perte KL, on combine deux KL avec $\alpha = 0.8$:

$$\mathcal{L}_{\text{KL}} = \alpha\, \text{KL}\big(\text{sg}(q) \,\|\, p\big) + (1 - \alpha)\, \text{KL}\big(q \,\|\, \text{sg}(p)\big). \tag{12}$$

$\alpha = 0.8$ signifie que *le prior apprend plus vite à imiter le posterior que l'inverse*, ce qui produit un modèle du monde plus précis en imagination.

5.4 4. Percentile normalization des returns

Avant de calculer l'avantage, on normalise les returns par l'écart inter-percentiles 5–95 :

$$A^{\text{norm}} = \frac{G^{\lambda} - p_5}{\max(p_{95} - p_5,\ 1)} - \frac{V - p_5}{\max(p_{95} - p_5,\ 1)}. \tag{13}$$

Plus robuste qu'une normalisation par moyenne / écart-type quand la distribution des returns est très asymétrique (cas du trading).

5.5 5. EMA target critic

Le critique *cible* $V_{\theta'}$ est une moyenne exponentielle ($\tau = 0.98$) des poids du critique en ligne : $\theta' \leftarrow \tau\,\theta' + (1-\tau)\,\theta$. Stabilise la cible des λ-returns, comme dans DQN.

5.6 6. Unimix exploration

La distribution de l'acteur est un mélange :

$$\pi(a \mid s) = (1-\varepsilon)\,\text{softmax}(\text{logits}) + \varepsilon \cdot \frac{1}{|\mathcal{A}|}, \quad \varepsilon = 0.01. \tag{14}$$

Garantit une probabilité minimale non nulle pour chaque action et empêche l'effondrement de la politique.

6 Application au *trading* crypto

6.1 Environnement

— **Données.** `SOLUSDT-aggTrades-2026-02.csv` (Binance, aggTrades).

— **Volume bars.** On ne raisonne plus en pas de temps mais en *barres de volume* (`volume_threshold=1000`). Cela uniformise l'information par barre.

— **Observation** $o_t \in \mathbb{R}^{121}$**.** Fenêtre de 30 dernières barres avec : prix de clôture, plus haut, plus bas, volume (chacun normalisé), plus 1 scalaire d'inventaire.

6.2 Actions discrètes

Trois actions, mappées vers l'environnement continu sous-jacent :

Index	Action	Effet
0	HOLD	Aucune position prise
1	BUY	Achat de *quantity_per_trade* = 5 unités
2	SELL	Vente de *quantity_per_trade* = 5 unités

6.3 Récompense

La récompense provient de l'environnement `ContinuousBinanceEnv` : ΔPnL (Profit and Loss) entre deux barres, ajusté de pénalités d'inventaire pour décourager les positions extrêmes.

6.4 Inférence en production

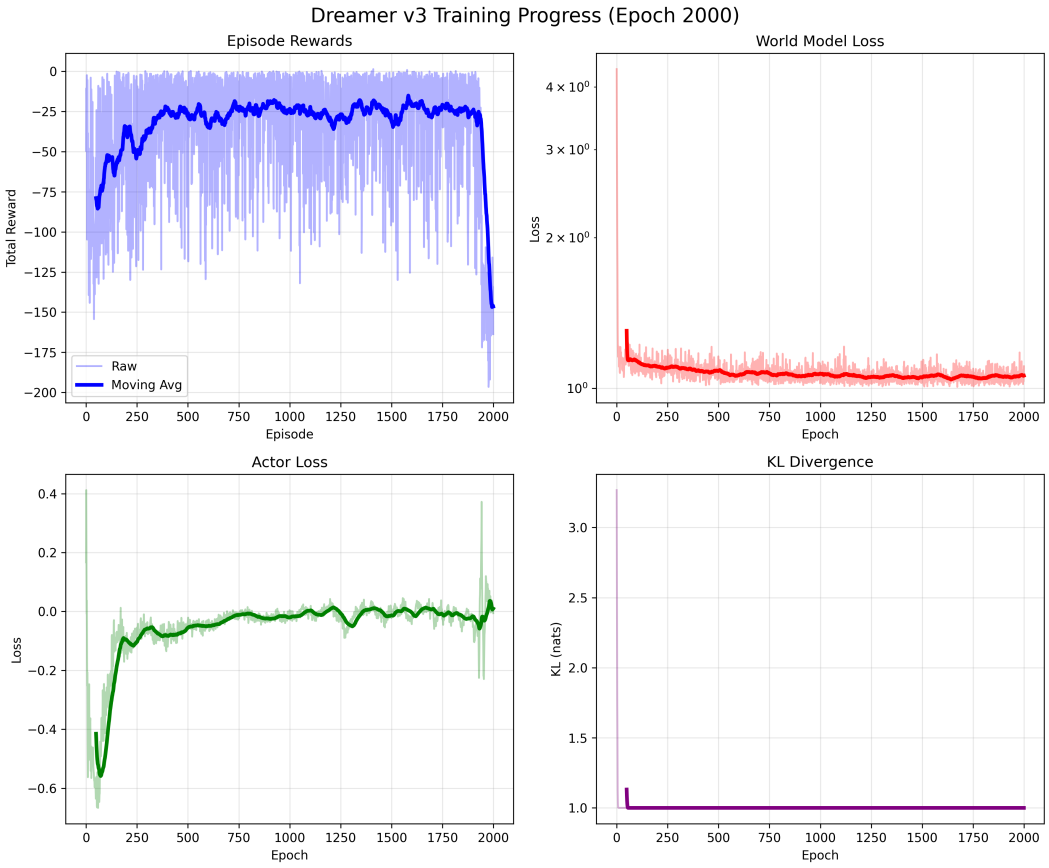
La classe `DreamerStrategy` (`agents/strategies/dreamer_agent.py`) implémente `BaseStrategy` :

— Au démarrage, charge `models/dreamer_agent.pth`.

— À chaque tick (`update_data` + `compute_signal`) :

1. Construit l'observation à partir des 30 derniers prix.
2. Encode + $RSSM$.observe_step (état persistant entre les appels).
3. Politique **greedy** (arg max) en inférence, pas d'échantillonnage.
4. Renvoie un `StrategySignal` (BUY / SELL / HOLD) avec une *confidence* = probabilité associée à l'action choisie.

6.5 Résultats



Quatre courbes : rewards par épisode, perte du modèle du monde (échelle log), perte de l'acteur, KL divergence. Le modèle du monde converge typiquement en quelques centaines d'époques ; l'acteur prend plus de temps puisqu'il s'entraîne sur les rollouts imaginaires.

7 Forces et limites

7.1 Forces

— **Efficacité en données** – une trajectoire réelle alimente des centaines de rollouts imaginaires.

— **Latents catégoriels** – expressifs et stables.

— **Hyperparamètres robustes** – les défauts Dreamer v3 v3 fonctionnent presque tels quels sur les données crypto.

— **Modèle du monde réutilisable** – on peut interroger le modèle : « que se passerait-il si j'achetais maintenant ? ».

7.2 Limites

— **Coût d'entraînement.** Plus lourd que DQN ou PPO : 7 réseaux à entraîner, GRU récurrente, rollouts imaginaires.

— **Risque de *model bias*.** Si le modèle du monde apprend des dynamiques fausses, l'acteur les exploite – d'où l'importance du KL balancing et des free bits.

— **Non-stationnarité du marché.** Le modèle entraîné en février 2026 peut dériver en mars ; un ré-entraînement glissant serait nécessaire en production.

8 Comparaison avec les autres agents du projet

Agent	Type	Modèle ?	Politique	Action	Particularité
DQN	off-policy	non	déterministe (greedy)	discrète	Replay + target
A2C	on-policy	non	stochastique	discrète	Avantage simple
PPO	on-policy	non	stochastique	discrète	Clip ratio
SAC	off-policy	non	stochastique	continue	Max-entropy
Dreamer v3	**model-based**	**oui**	stochastique	discrète	Imagination

Dreamer v3 est *le seul agent du projet à apprendre une dynamique explicite*. Tous les autres apprennent uniquement π et / ou Q.

9 Code clé : extraits commentés

9.1 Boucle d'entraînement (cœur)

```
# 1. Collecte d'un épisode reel
while not done:
    embed = encoder(obs)
    h, z, _, _ = rssm.observe_step(h, z, prev_action, embed)
    state = torch.cat([h, z], dim=-1)
    action_idx = torch.multinomial(actor(state), 1).item()
    next_obs, reward, done = env.step(action_idx)

# 2a. Entrainement du modele du monde sur sequences echantillonnees
wm_loss = l_recon + l_reward + l_continue + l_kl

# 2b. Imagination + acteur-critique
for t in range(H):
    state_t = torch.cat([cur_h, cur_z], dim=-1)
    a_t = sample(actor(state_t))
    r_t = symexp(reward_pred(state_t))
    cur_h, cur_z = rssm.imagine_step(cur_h, cur_z, a_t)
```

9.2 Calcul des λ-returns (rétropropagation arrière)

```
returns[-1] = imag_rewards[-1] + gamma * imag_continues[-1] * values_ema[-1]
for t in reversed(range(H - 1)):
    returns[t] = imag_rewards[t] + gamma * imag_continues[t] * (
        (1.0 - lam) * values_ema[t + 1] + lam * returns[t + 1]
    )
```

10 Questions probables en soutenance (& réponses)

Pourquoi Dreamer v3 et pas un PPO ?
PPO est *model-free* : chaque trajectoire vue est consommée directement. Dreamer v3 apprend un

modèle du monde réutilisable, ce qui améliore l'efficacité en données – crucial quand on n'a qu'un mois de ticks Binance par paire.

Pourquoi des latents catégoriels et pas gaussiens ?
Empiriquement (DreamerV2/V3), les catégorielles sont plus stables numériquement, n'ont pas le problème d'effondrement de variance des gaussiennes, et se prêtent à des KL exacts.

Que sont h_t et z_t ?
h_t est la *mémoire déterministe* (sortie de la GRU) ; z_t est l'*état stochastique* (variables catégorielles échantillonnées). L'état complet est $s_t = [h_t, z_t]$.

Pourquoi le straight-through estimator ?
Parce qu'on ne peut pas dériver à travers un arg max. On utilise forward = one-hot, backward = gradient des softmax. C'est un biais contrôlé qui marche bien en pratique.

Que se passe-t-il si le modèle du monde est mauvais ?
L'acteur apprend dans une simulation incorrecte ⇒ il exploitera des artefacts inexistants dans le vrai marché. C'est le *model bias*. On le combat avec : (a) le KL balancing (le prior se rapproche du posterior), (b) les free bits (information minimale dans z), (c) la collecte continue (le modèle revoit régulièrement la vraie distribution).

Pourquoi symlog ?
Les récompenses crypto peuvent varier de 10^{-3} à 10^{2}. Sans compression, un seul outlier domine la perte MSE. symlog préserve le signe et compresse les magnitudes.

Pourquoi un EMA target critic ?
La cible des λ-returns dépend de V. Si la cible bouge à chaque pas, l'entraînement diverge (problème classique du DQN). On gèle une copie qui suit lentement le critique en ligne.

L'horizon $H = 15$ est-il suffisant ?
C'est un compromis. Plus grand ⇒ plus d'erreur cumulée du modèle. Plus petit ⇒ le critique doit absorber plus de crédit. $H = 15$ avec $\gamma = 0.997$ couvre des effets non négligeables ($0.997^{15} \approx 0.96$).

Pourquoi un buffer *séquentiel* et pas un classique transition replay ?
Le RSSM est récurrent : il a besoin de *séquences contiguës* pour dérouler la GRU et apprendre la dynamique. On échantillonne donc des sous-trajectoires, pas des transitions isolées.

Comment évaluez-vous l'agent en production ?
Greedy (arg max) sur la sortie de l'acteur, avec un état RSSM maintenu entre les appels. La confiance de la décision est la probabilité softmax de l'action choisie.

Quelle est la complexité d'un pas d'entraînement ?
Par pas de gradient : $B \cdot T$ appels au RSSM en posterior ($16 \cdot 64 = 1024$) puis $B \cdot T \cdot H$ appels en imagine ($1024 \cdot 15 \approx 15\,360$). Sur GPU, c'est dominé par les multiplications matricielles $1536 \to 512$ du décodeur et de l'acteur.

Que fait `prev_action` et pourquoi un zéro au premier pas ?
La GRU prend en entrée $[z_{t-1}, a_{t-1}]$ – l'action qui a *conduit* à l'observation courante. À $t = 0$, il n'y a pas d'action précédente, donc on injecte un vecteur nul comme marqueur « aucune action ». C'est cohérent entre collecte, entraînement et inférence.

11 Pour aller plus loin

- **Référence principale.** Hafner, Pasukonis, Ba, Lillicrap. *Mastering Diverse Domains through World Models*. arXiv 2301.04104, 2023.
- **Code dans le projet.**
  - `agents/strategies/dreamer_agent.py` – réseaux et stratégie d'inférence (`DreamerStrategy`).
  - `agents/train_dreamer.py` – boucle d'entraînement complète.
  - `run_dreamer.py` – script de lancement.

  - `models/dreamer_agent.pth` – checkpoint sauvegardé.
- **Pistes d'amélioration.**
  - Étendre les actions à un espace continu (taille de position variable).
  - Conditionner le RSSM sur des features de carnet d'ordres (profondeur, déséquilibre bid/ask).
  - Ré-entraîner en *rolling window* pour gérer la non-stationnarité.